From MapReduce to Lakehouse

How Query Engines connect both end?

What to expect?

- **Try to tell a binding story**
- Motivate the case for Lakehouse and most importantly understand the lead up to it
- It is not an exhaustive list
- Can we summarize and yet create a minimal set of concepts
- About myself:
  - Joydeep Banik Roy
    - Director, Data Science & ML Eng at Zeotap CDP
    - Worked in Data Engineering and Data Science for 13+ yrs
    - Actively involved in the Spark + Delta Lake Community

An evolving dataset problem

Several partners deliver daily customer and event datasets. You must clean them, join them to reference data, calculate audience metrics, and publish a table for analysts. Delivery sizes vary, some records arrive late, and partners sometimes resend corrected files.

Can we process it efficiently?

Parallelism, input layout, joins, skew.

Can we answer questions efficiently?

Query planning, execution, approximate aggregates.

Can we publish and update it reliably?

Retries, concurrent readers, corrections, reproducibility

“Everybody eventually wants a DB which performs within a SLA that comes at a very low cost while providing performance guarantees”

What are the choices we have?

Takes care of both Storage and Query

PostgreSQL

- Transactional; great
- Very high cost for TBs (or PBs) of data

HBase or Cassandra or Aerospike

- Data Modelling Challenges
  - Store as Key Value
- Still high cost

Storage

- Cheap distributed storage
  - Google Cloud Storage
  - AWS S3

Query Engines

- Needs to be queried based on need
  - OLTP
  - OLAP
- **Apache Spark**, Facebook Presto,Google BigQuery, Amazon Athena, TrinoDB

Landscape Review

Step	What becomes easier	What remains difficult
MapReduce	Distributing computation with a constrained programming model	Expressing and coordinating multi-step pipelines
Higher-level pipeline APIs, including Crunch and Beam	Describing transformations and their dependencies	Execution costs and data distribution
Spark	Executing richer computation graphs	Choosing operations and managing expensive movement
Spark SQL / DataFrames	Letting an optimizer reason about relational operations	Statistics, skew, layout, and runtime conditions
Lakehouse table formats	Managing reliable tables over files	Workload design, execution tuning, and operational discipline

Apache Spark

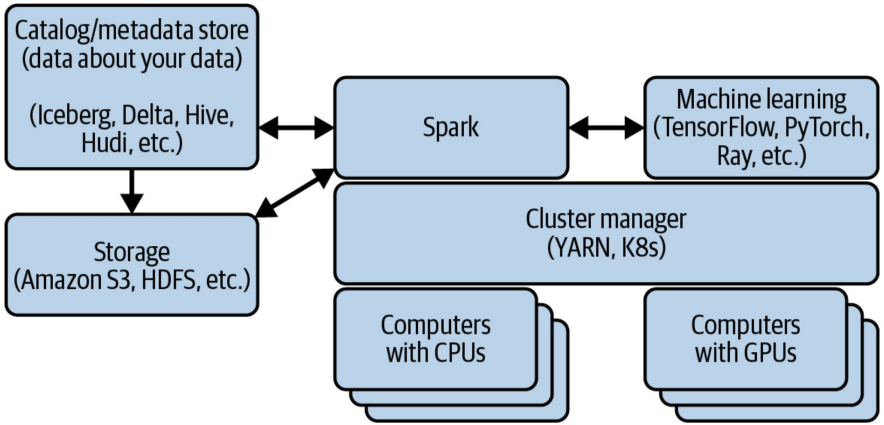
It's a query engine with in-memory runtime

Provides methods to process data

- Parallelly

You can run it

- On a single machine with single JVM
- Distributed storage systems like HDFS, Amazon s3 or Cassandra
- Catalog stores like Iceberg or Hive
- Cluster managers
  - Kubernetes or YARN
  - Databricks or Snowflake (Proprietary)



Apache Spark

RDD

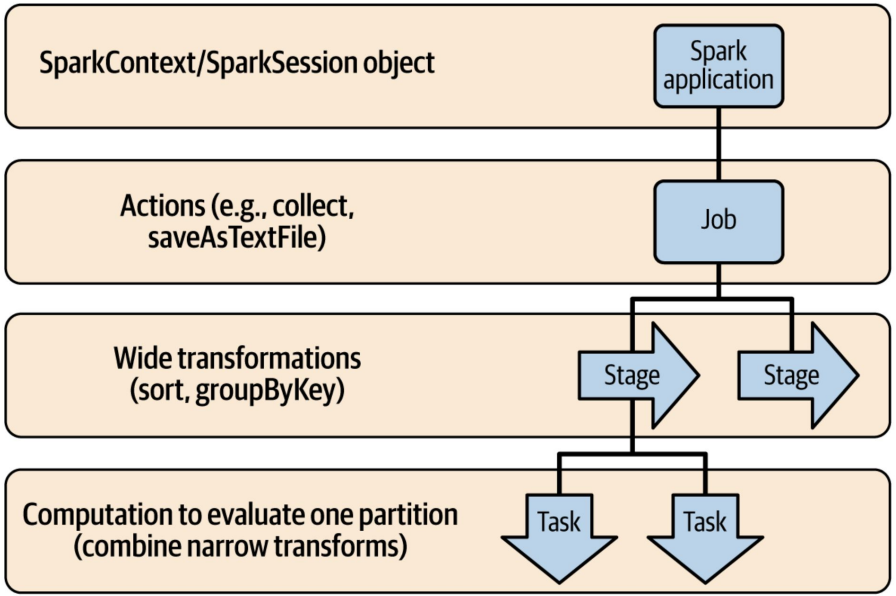
Lazily evaluated, statically typed, distributed collections

Dataset and Dataframe API

Strongly typed collection + relational and functional transformations

Spark SQL

Narrow vs Wide Transformations

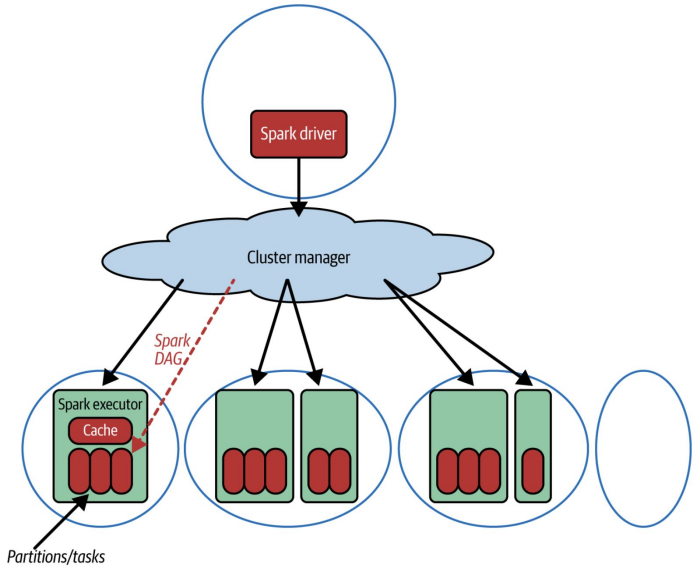


Executor cores < No. of partitions in the data?

Executor cores < No. of partitions in the data

It Depends!

- Unavailable Executors in cluster
- Rerun with more executors
    - Logic :
        - 1 partition
        - 1 task
        - 1 core



CSV or Parquet or Avro?

CSV vs Parquet vs Avro?

- Splittable vs non splittable
- Is CSV splittable?
- Schema Evolution?
  - Avro
- Columnar format advantage
  - Parquet

It depends!

Input	Can Spark split one file across reading tasks?
Uncompressed CSV, one record per line	Yes
CSV read with `multiLine=true`	No
Single gzip-compressed CSV	No
Multiple non-splittable CSV files	Can distribute different files across tasks

map vs mapPartitions?

map vs mapPartitions

It depends!

- Ideally if you want to do iterator level operations
  - Initialize a variable, lets say a huge HashMap once and then reuse it across partitions
  - If you can combine logic for a partition, then yes
- **Minimize object creation** in MAP and overall

Feature	`map()`	`mapPartitions()`
Scope	One record per function call	All records in a single partition per function call
Execution Count	Called N times (where N = total records)	Called P times (where P = total partitions)
Memory Footprint	Low and predictable per record	Higher (risk of Out-Of-Memory if you collect the whole iterator into memory)
Code Complexity	Simple lambda or function	Requires handling an iterator and yielding results

More questions

1. Dataset API vs Dataframe API?
2. Dataset vs Spark SQL?
3. JDBC Source vs Parquet Source?
4. Explode followed by map followed by reduce, should we rewrite?
5. Static vs Dynamic Allocation?
6. Should you partition your data, say by country?

More prescriptions

1. Dataset API vs Dataframe API?
2. Dataset vs Spark SQL?
3. JDBC Source vs Parquet Source?
4. Explode followed by map followed by reduce, should we rewrite?
5. Static vs Dynamic Allocation?
6. Should you partition your data, say by country?

1. Preferably Dataset due to compile time type checking
2. It depends, SQL is highly reusable, you can write a pipeline in BigQuery and resume in Spark
3. If predicate push down is available
4. This one is trick question: Yes, we should, it does not depend
5. Dynamic Allocation unless you know exact configs
6. It depends, partitioning generally improves read and how files are loaded into executor memory but resulting file sizes should be reasonable

Join Strategies

Join Types	How it works	When it's used
Broadcast Hash Join	Spark copies (broadcasts) the smaller DataFrame to all worker nodes, avoiding a costly cluster-wide data shuffle	When one DataFrame is smaller than the configured threshold (defaulting to 10MB) and an equi-join (=) condition is used
Sort Merge Join	A two-step process where Spark first shuffles data across the cluster so matching keys land on the same partitions, sorts the partitions by the key, and then merges them sequentially	This is the default strategy for large-to-large equi-joins where neither table fits into memory
Shuffle Hash Join	Shuffles both datasets so identical keys end up in the same partition. Spark then builds an in-memory hash table out of the smaller partition to match with the larger one	Used as an alternative to Sort Merge Join for equi-joins when data is relatively evenly distributed, but sorting would add too much overhead

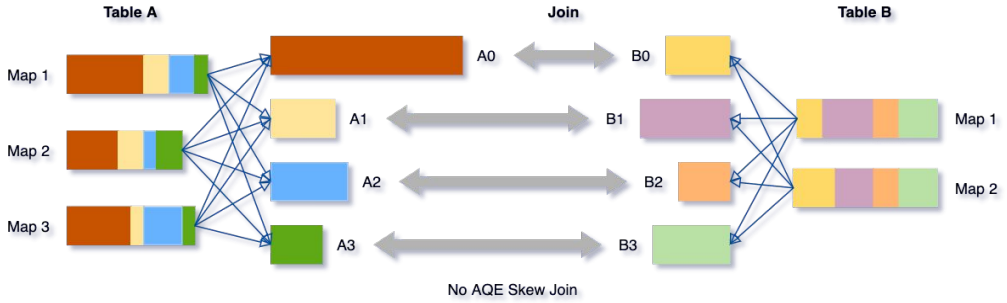
Managing the Skew

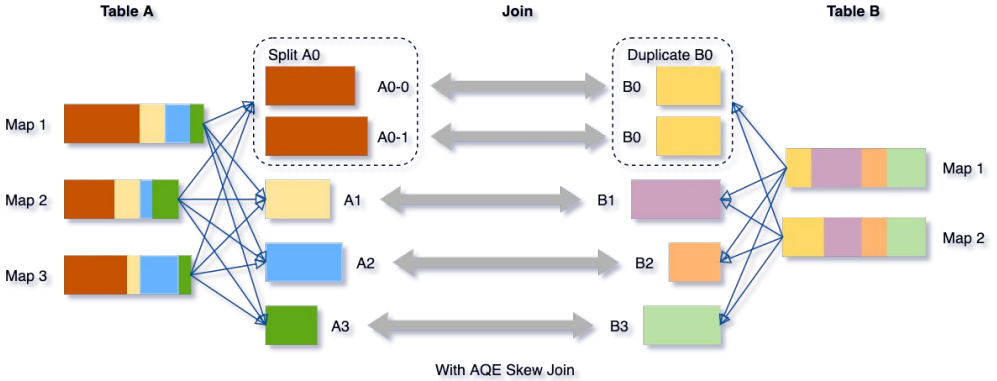
Enable Adaptive Query Execution

Salting : It works by breaking up a single heavy, overloaded key into multiple distinct sub-keys

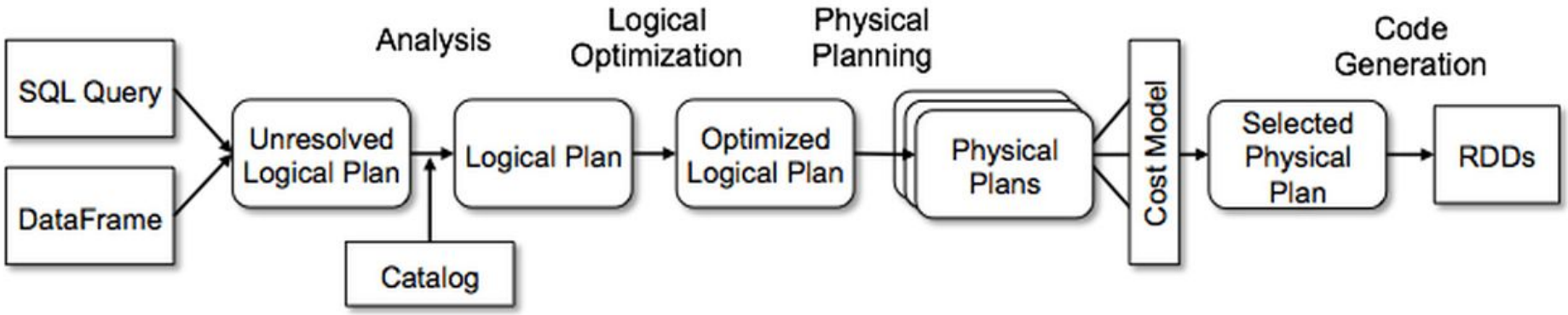
Hot Key Isolation

Bucketing





Spark Query Planning & Optimizer



Accelerators

- Databricks Photon (Closed Source)
- Apache Arrow + Datafusion Comet
- Project Gluten
  - Uses Apache Arrow
  - Supports Arrow Compute, Facebook Velox and Clickhouse
- Spark RAPIDS
  - For GPU acceleration

Open source and closed source accelerators have improved the running speed of your existing spark code without rewriting it

Approximations and Sketches

Question	What to use	Gotchas
Average	Sum and count	Averaging partition averages is wrong when partition sizes differ.
Median / percentile	A suitable quantile sketch for an approximate answer	Partition medians alone do not determine the global median.
Unique users	HyperLogLog sketch	Daily unique counts cannot simply be added; compatible sketches can be merged.
Membership	Bloom filter	A negative can rule out membership; a positive needs verification when exactness matters.

Files, incremental updates, and production correctness

When does a directory of files stop being enough?

“Yesterday’s records have changed. We need to replace some data while analysts are querying the table. The job fails halfway through.”

- Which files constitute the current table?
- What should a reader see during the update?
- Can we retry without duplicating logical records?
- What happens if another writer modifies overlapping data?
- Can we reproduce the result an analyst saw yesterday?

Those questions make transaction logs, snapshots, concurrency control, and versioning feel necessary.

QUESTIONS?

THANK YOU